

SQL & MySQL

A Complete Beginner's Guide

From Database Fundamentals to Writing SQL Queries

Compiled and explained from personal study notes

1. Introduction to Databases

1.1 Data vs Information

Before understanding databases, it is important to understand the difference between raw data and information.

Term	Meaning	Example
Raw Data	Facts and figures with no context on their own.	"Krish 500"
Information	Data that has been processed or given context so it becomes meaningful.	"Krish earns 5000"

Without a database, it becomes very difficult to efficiently perform CRUD operations (Create, Read, Update, Delete) on data. This is exactly the problem a database solves.

1.2 What is a Database (DB)?

A database is basically a container — an organized system used to store, manage, and retrieve data efficiently and reliably. Instead of scattering data across files, a database keeps it structured so it can be searched, updated, and secured easily.

1.3 Types of Databases (Structural vs Non-Structural)

Structural (Structured)	Non-Structural (Unstructured / Semi-structured)
Data is organized in a fixed, predefined format (rows and columns), e.g. relational databases.	Data does not follow a fixed format — e.g. documents, JSON, images, videos, key-value pairs used by NoSQL databases.

1.4 Ways to Classify Databases

- Based on the Data Model — how the data is structurally organized (RDBMS, NoSQL, Hierarchical, Object-Oriented, Network DB).
- Based on Usage / Development — the purpose the database serves in an application (transactional systems, analytical systems, etc.).

2. Types of Databases Based on Data Model

Type	How Data is Stored	Examples	Typical Use Case
RDBMS	Rows & Columns (tables)	MySQL, Oracle	Banking, ERP systems

Type	How Data is Stored	Examples	Typical Use Case
NoSQL	Documents, key-value pairs, column families	MongoDB, Redis	Social media, Big Data
Hierarchical DB	Tree structure (parent-child)	IBM IMS, XML	Old mainframe systems
Object-Oriented DB	Objects, similar to OOP concepts	db4o	Gaming, multimedia, CAD
Network DB	Graph-like structure with multiple links	IDMS	Old telecom systems

2.1 RDBMS — Relational Database Management System

RDBMS stands for Relational Database Management System. Its structure is based on tables that are linked together using Primary Keys and Foreign Key relationships.

- SQL (Structured Query Language) — the standard language used to communicate with an RDBMS. It follows a standardized syntax across most relational databases.
- NoSQL ("Not Only SQL") — used when data is huge in volume and unstructured in nature. It does not strictly require a fixed schema.

2.2 Why is RDBMS often preferred?

- Less data duplication because of normalization.
- High data integrity, since relationships and constraints are enforced.
- Higher security controls.
- Supports a large number of concurrent users reliably.

2.3 Relational DB vs Non-Relational DB

Feature	Relational Database	Non-Relational (NoSQL) Database
Structure	Structured — organized into tables	Semi-structured / unstructured
Example	MySQL	MongoDB
Query Language	SQL	NoSQL, JSON-based, CQL, etc.
Joins	Supported	Not supported (generally)
Schema	Pre-defined (fixed)	Dynamic and flexible
Scaling	Vertical scaling — needs bigger servers	Horizontal scaling — add more servers
Transactions	ACID compliant	BASE (Basically Available, Soft state, Eventually consistent)
Data Integrity	High — strong constraint support	Lower — constraints often not enforced

Feature	Relational Database	Non-Relational (NoSQL) Database
Speed	Great for complex queries with relationships	Very fast for simple reads & writes

Scaling: Vertical vs Horizontal

- Vertical Scaling — increasing the capacity of a single server (more RAM, CPU, storage).
- Horizontal Scaling — adding more servers/machines to share the load.

2.4 NoSQL Sub-types (Based on Structure)

Sub-type	Description	Example
Document DB	Stores data in JSON-like documents (key-value pairs inside a document).	MongoDB
Key-Value DB	Works like a dictionary — every value is accessed through a unique key.	Redis
Column DB	Stores data column-wise instead of row-wise; columns are stored together.	Cassandra
Graph DB	Stores data as nodes and relationships (edges), ideal for connected data.	Neo4j

2.5 DBMS vs RDBMS

DBMS	RDBMS
No normalization enforced	Follows normalization rules
No relationships between data	Relationships via Primary Key & Foreign Key
Example: XML file-based systems	Example: MySQL, PostgreSQL

Note: DBMS is the general umbrella term for any software that manages databases. RDBMS is a specific type of DBMS that stores data in structured tables and enforces relationships.

3. Data Types in SQL

A data type defines the kind of value that can be stored in a particular column of a table (for example, whole numbers, decimals, text, or dates). Choosing the right data type keeps data accurate and storage efficient.

SQL data types are broadly grouped into four categories:

- Numeric Data Types
- String Data Types
- Date & Time Data Types
- Boolean Data Type

3.1 Numeric Data Types

Data Type	Description
INT	Stores whole numbers (integers). Takes up 4 bytes of storage.
TINYINT	A smaller integer type, takes up only 1 byte.
DECIMAL(M, D)	Stores exact fixed-point numbers. M = total number of digits, D = number of digits after the decimal point. Example: DECIMAL(5,2) can store values up to 999.99.

3.2 String Data Types

Data Type	Description
VARCHAR(size)	Stores variable-length text; memory is allocated based on the actual length of the text entered, up to the defined size.
CHAR(size)	Stores fixed-length text; memory is always allocated for the full defined size, regardless of the actual text length.
TEXT / BLOB	Used to store large amounts of text (Binary Large Object). TEXT can typically store up to about 65,000 characters.

Note: Use CHAR for fields with a consistent length (like a 2-letter country code) and VARCHAR for fields whose length varies (like a name or an email address).

3.3 Date & Time Data Types

Data Type	Format	Description
DATE	YYYY-MM-DD	Stores only the date.
DATETIME	YYYY-MM-DD HH:MM:SS	Stores both date and time.
TIMESTAMP	YYYY-MM-DD HH:MM:SS	Stores date and time, and can automatically update itself when the row is modified.

3.4 Boolean Data Type

Used to store logical values — true or false (in MySQL this is often internally represented as TINYINT(1), where 1 = true and 0 = false).

4. Choosing a Database: MySQL vs Oracle

Criteria	MySQL	Oracle
Cost	Free (open-source)	Paid (licensed)
Flexibility	Flexible and easy to work with	Feature-rich but more rigid setup
Ease of Use	Beginner-friendly	Steeper learning curve
Usage Limits	Fewer restrictions for common use	More enterprise-grade limits/controls

Note: This is a simplified comparison from the notes. In practice the right choice depends on scale, budget, and the specific features an organization needs — both are powerful, production-grade RDBMS platforms.

5. Constraints

Constraints are a set of rules applied to columns in a table to validate and control what data can be entered. They help maintain the accuracy and reliability of data inside the database.

Constraint	Purpose
NOT NULL	Ensures a column cannot have a NULL (empty) value.
UNIQUE	Ensures all values in a column are different from one another. A column with only a UNIQUE constraint can still accept one NULL value.
PRIMARY KEY	A special constraint that is both UNIQUE and NOT NULL. It uniquely identifies each row in a table. A table can have only one primary key.
FOREIGN KEY	Used to establish a relationship between two tables. A table can have multiple foreign keys, each referencing a primary key in another table.
CHECK	Used to check that values meet a specific condition before being inserted. Multiple conditions can be combined into a single CHECK constraint.
DEFAULT	Automatically assigns a default value to a column when no value is provided during insertion.

```
-- Example: constraints in a CREATE TABLE statement
CREATE TABLE emp (
  empid    INT PRIMARY KEY,
  ename    VARCHAR(50) NOT NULL,
  email    VARCHAR(100) UNIQUE,
  sal      DECIMAL(8,2) DEFAULT 0,
  deptno   INT,
  CHECK (sal >= 0),
  FOREIGN KEY (deptno) REFERENCES department(deptno)
);
```

6. ER Diagram / ER Model

The Entity Relationship (ER) Model is a conceptual blueprint of a database. It represents entities, their attributes, and the relationships (connections) between them. The ER model is used before schema design to bring clarity to how the data will be organized.

6.1 Key Components

Component	Meaning	Represented As
Entity	Anything from the real world that data is being stored about (e.g. Student, Employee, Department, Course).	Rectangle
Attribute	A property or characteristic of an entity (e.g. empid, ename, job).	Oval
Relationship	An association between two or more entities (e.g. "Students choose Courses").	Diamond

6.2 Cardinality

Cardinality describes how many instances of one entity relate to instances of another entity.

- 1 : 1 — One instance relates to exactly one instance of another entity.
- 1 : N — One instance relates to many instances of another entity.
- N : N (Many-to-Many) — Requires an intermediate (junction) table to be created to manage the relationship.

Note: A foreign key can be taken from any of the related tables to represent a 1:1 or 1:N relationship, but for N:N relationships, an intermediate/junction table is required.

6.3 Example: Employee – Department

Entities: Employee and Department (related in a 1 : M relationship — one department has many employees).

- Employee attributes: empid, ename, job
- Department attributes: deptno, dname, loc

7. Database Schema

A schema is the logical container or blueprint for database objects. It defines the tables, columns, data types, and constraints — essentially acting as a map of how data is stored and accessed.

7.1 Why do we need a Schema?

- To avoid redundancy (duplicate/repeated data).

- To support normalization.
- To give a clear overview of the entire database structure.

7.2 Components of a Schema

- Tables — store the data using rows and columns.
- Columns — define the attributes, along with their data type and constraints.
- Rows — the individual records; constraints are the rules applied on them.

7.3 ER Model vs Schema

ER Model	Schema
Conceptual design	Physical design
Shows entities, attributes, and relationships	Defines the actual tables, columns, and constraints
Used for planning	Used for implementation

8. Types of SQL Statements

SQL statements are grouped into five categories, based on the kind of task they perform:

#	Category	Full Form	Purpose
1	DDL	Data Definition Language	Defines and modifies the structure of database objects.
2	DML	Data Manipulation Language	Manipulates the actual data stored inside tables.
3	TCL	Transaction Control Language	Manages transactions within the database.
4	DCL	Data Control Language	Controls access/permissions to the database.
5	DQL	Data Query Language	Used to fetch/retrieve data from the database.

8.1 DDL — Data Definition Language

Commands: CREATE, ALTER, RENAME, DROP, TRUNCATE

Command	Description
CREATE	Creates a new database or table.
ALTER	Modifies the structure of an existing database/table.
DROP	Permanently deletes a table or database, including its structure.
TRUNCATE	Removes all rows from a table in one shot, but keeps the table structure.
RENAME	Renames a database object (e.g. a table).

DROP vs TRUNCATE vs DELETE

This is a common point of confusion for beginners, so it's worth understanding clearly:

- DELETE removes rows one record at a time, which makes it slower — this is why it is not recommended for clearing large tables.
- TRUNCATE deletes all records in one shot (much faster) but keeps the table structure intact. It cannot be used with a WHERE clause.
- DROP removes the table (or database) permanently, structure included — it no longer exists afterward.

8.2 DML — Data Manipulation Language

Commands: INSERT, UPDATE, DELETE

Command	Description
INSERT	Adds new rows of data into a table.

Command	Description
UPDATE	Modifies existing data within a table.
DELETE	Removes specific rows from a table, based on a condition.

8.3 TCL — Transaction Control Language

Commands: COMMIT, ROLLBACK, SAVEPOINT

Command	Description
COMMIT	Saves all the changes made during the current transaction permanently.
ROLLBACK	Undoes changes made during the current transaction process.
SAVEPOINT	Sets a point within a transaction to which you can roll back later, if needed.

8.4 DCL — Data Control Language

Commands: GRANT, REVOKE

Command	Description
GRANT	Gives specific access permissions to a database user.
REVOKE	Takes away previously granted permissions from a user.

8.5 DQL — Data Query Language

The DQL command is SELECT — used to fetch and retrieve data.

Term	Description
SELECT	Selects data and displays it on the screen.
Projection	Used to get specific column(s) — i.e. controlling which columns are shown.
Selection	Used to filter which rows are returned, based on a condition (can involve both rows and columns).
Join	Used to combine and fetch data from two (or more) tables simultaneously.

8.6 CRUD Operations

CRUD is a common acronym summarizing the four basic operations performed on data — these map directly onto DDL/DML commands:

Letter	Operation	Corresponding SQL
C	Create / Insert	INSERT (DML), CREATE (DDL)
R	Read / Retrieve	SELECT (DQL)
U	Update / Modify	UPDATE (DML)
D	Delete	DELETE (DML), DROP (DDL)

9. The SELECT Statement

The general syntax of a SELECT query is:

```
SELECT * | DISTINCT column_name | expression [alias]
FROM table_name
WHERE <filter_conditions>;
```

- The WHERE clause is used to filter records based on a condition.
- SQL values/keywords are not case-sensitive — queries can be written in any case.
- A clause is a specific keyword that performs a specific task within a query (e.g. WHERE, FROM, GROUP BY).

9.1 Projections

Projection refers to selecting specific columns from a table. The asterisk (*) can be used alone, or as emp.* — both work the same way and return all columns.

```
SELECT * FROM emp;
```

DISTINCT

DISTINCT is used to return only the unique values/records from the given column(s).

```
-- Returns each unique salary value
SELECT DISTINCT sal
FROM emp;

-- Returns unique combinations (paired) of salary and job
SELECT DISTINCT sal, job
FROM emp;
```

Expressions and Aliases

An expression combines operands with an operator — for example, sal + 200. An alias (AS) is used to give a temporary, more readable name to a column or expression in the output.

```
SELECT sal + 200 AS bonus_sal
FROM emp;

-- AS is optional
SELECT sal + 200 bonus_sal
FROM emp;
```

9.2 Selections (Filtering with WHERE)

Example: "Write a query to display names of employees who were hired after 1981."

```
SELECT ename
FROM emp
WHERE hiredate > '1981-12-31';

-- Alternative using YEAR() function
SELECT ename
FROM emp
WHERE YEAR(hiredate) > 1981;
```

Example: "Write a query to display name and job of employees who are not a clerk."

```
SELECT ename, job
FROM emp
WHERE job = 'CLERK';    -- equals

SELECT ename, job
FROM emp
WHERE job <> 'CLERK';    -- not equal to
```

Note: Multiple conditions can be combined inside a WHERE clause using logical operators such as AND, OR, and NOT.

10. Operators in SQL

Operators are symbols that perform specific operations on values or expressions. SQL has several categories of operators:

Category	Examples
Arithmetic Operators	+, -, *, /, %
Relational / Comparison Operators	=, <>, >, <, >=, <=
Concatenation Operator	(or CONCAT() in MySQL)
Logical Operators	AND, OR, NOT
Special Operators	IN, NOT IN, BETWEEN, NOT BETWEEN, IS, IS NOT, LIKE, NOT LIKE
Subquery Operators	ALL, ANY, EXISTS, NOT EXISTS

10.1 Special Operators

IN / NOT IN

Used to check whether a value matches any value within a given list.

```
SELECT name
FROM emp
WHERE ename IN ('SMITH', 'MOHAN');
```

BETWEEN

Used for a range of values. The BETWEEN operator is inclusive of both boundary values, and the range order cannot be swapped (the smaller value must come first).

```
SELECT * FROM emp WHERE sal BETWEEN 3000 AND 6000;
```

IS / IS NOT (NULL checks)

The equality operator (=) cannot be used to compare with NULL, because NULL represents an unknown value, not a comparable one. The IS operator must be used along with the NULL keyword to check for NULL values. IS NOT is used to reject/exclude NULL values.

Example: "Write a query to display names of employees who do not have any reporting manager."

```
SELECT ename
FROM emp
WHERE mgr IS NULL;
```

Example: "Display name, job, and salary of employees who have a reporting manager and belong to department 30."

```
SELECT ename, job, sal
FROM emp
WHERE mgr IS NOT NULL AND deptno = 30;
```

LIKE / NOT LIKE (Pattern Matching)

LIKE is used for pattern-based searches on string values, using wildcard characters: % (any number of characters) and _ (exactly one character).

Pattern	Meaning
'%R%R%'	Contains the letter R at least twice
'%LL%'	Contains two consecutive L's
'S%T'	Starts with S and ends with T
'_A%'	Has 'A' as the second character
'%E_'	Has 'E' as the second-to-last character
'____'	Exactly 4 characters long (four underscores)

10.2 Subquery Operators

ANY

The ANY operator selects records that are true for any one of the values returned by a subquery. It works similarly to the OR operator. Common forms: > ANY, < ANY, >= ANY, <= ANY.

ALL

The ALL operator selects records that are true for every value returned by the subquery — it works similarly to the AND operator. Common forms: > ALL, < ALL, >= ALL, <= ALL.

10.3 Combining Multiple Conditions

Example: "Display names, job, and deptno of employees who work as CLERK and belong to either department 10 or 30."

```
SELECT ename, job, deptno
FROM emp
WHERE job = 'CLERK' AND (deptno = 10 OR deptno = 30);
```

11. Functions in SQL

A function is a block of code / a set of instructions that performs a specific task and returns a result.

11.1 Types of Functions

- Predefined (Built-in) Functions
 - • Single-row functions — take multiple inputs and return one output per row (e.g. UPPER(), ROUND()).
 - • Multi-row (Aggregate/Group) functions — take multiple input rows and return a single, combined output.
- User-Defined Functions — custom functions created by the user for reusable logic.

11.2 Aggregate (Group) Functions

These functions work only on numerical data (except COUNT, which can work on any data type):

Function	Description
MAX()	Returns the highest value in a column.
MIN()	Returns the lowest value in a column.
SUM()	Returns the total sum of values in a column.
AVG()	Returns the average of values in a column.
COUNT()	Returns the number of rows.

- COUNT(column_name) — ignores NULL values in that column.
- COUNT(*) — counts all rows, including those with NULLs.

Example: "Display the most recent hire date from the emp table."

```
SELECT MAX(hiredate) FROM emp;
```

12. Important Clauses

12.1 GROUP BY

GROUP BY is used to group similar records together, typically so an aggregate function can be applied to each group. It works row-by-row and, if any other clause executes after it, that clause runs group-by-group.

Example: "Write a query to display the sum of salary for each job."

```
SELECT job, SUM(sal)
FROM emp
GROUP BY job;
```

12.2 HAVING

HAVING is used to filter records after grouping — it filters groups, whereas WHERE filters individual rows before grouping.

Example: "Write a query to display duplicate salaries from the employee table."

```
SELECT sal
FROM emp
GROUP BY sal
HAVING COUNT(*) > 1;
```

12.3 ORDER BY

ORDER BY is used to arrange the records in a particular order — ascending or descending. By default, sorting is in ascending order. ASC = ascending, DESC = descending. When multiple columns are given, priority is given to the first field listed.

```
-- Sort by job, then by salary within each job
SELECT job, sal
FROM emp
ORDER BY job, sal;

-- Distinct salaries, sorted
SELECT DISTINCT sal
FROM emp
ORDER BY sal;
```

12.4 LIMIT

LIMIT is a clause used to restrict the number of records returned by a query. It should always be placed at the end of the query.

```
-- Without OFFSET
SELECT col_name
FROM table_name
LIMIT row_count;

-- With OFFSET (skips a number of records first)
SELECT col_name
FROM table_name
LIMIT offset, row_count;
```

Example: "Get the 2 earliest-hired employees."

```
SELECT * FROM emp
ORDER BY hiredate ASC
LIMIT 2;
```

13. Subqueries

A subquery is a query written inside another query (also called an inner query). It is often used when a condition depends on a value that must first be looked up from the database itself, rather than being known in advance.

13.1 Types of Subqueries

Type	Description
Single-Row Subquery	Returns exactly one value as its output. Can be used with =, >, <, etc.
Multi-Row Subquery	Returns more than one row/value as output. Must be used with IN, ANY, ALL, EXISTS.

13.2 Employee–Manager Relationships

A very common real-world subquery pattern involves employee-manager relationships, which typically requires two steps:

- Step 1: Identify the manager (or the relevant employee) using an inner query.
- Step 2: Use that result in the outer query to find the related employees.

Example: "Write a query to display the manager's name of FORD."

```
SELECT ename FROM emp
WHERE empno = (
    SELECT mgr
    FROM emp
    WHERE ename = 'FORD'
);
```

Example: "Display the maximum salary among employees who report to KING."

```
SELECT MAX(sal) FROM emp
WHERE mgr = (
    SELECT empno
    FROM emp
    WHERE ename = 'KING'
);
```

13.3 Subqueries Returning Multiple Rows — Using IN

When a subquery can return more than one value, the equality operator (=) cannot be used — IN must be used instead.

Example: "Display the name and location of the department where employees Allen and Jones work."

```

SELECT dname, loc
FROM dept
WHERE deptno = (
    SELECT deptno
    FROM emp
    WHERE ename IN ('ALLEN', 'JONES')
);
-- Note: use IN here, NOT '=', since the subquery
-- can return more than one row.

```

Example: "Display the number of employees working at the location 'CHICAGO'."

```

SELECT COUNT(*)
FROM emp
WHERE deptno = (
    SELECT deptno
    FROM dept
    WHERE loc = 'CHICAGO'
);

```

13.4 More Subquery Examples

Example: "Display details of employees whose salary is greater than Allen's salary."

```

SELECT * FROM emp
WHERE sal > (
    SELECT sal
    FROM emp
    WHERE ename = 'ALLEN'
);

```

Example: "Display name and job of employees hired after James, who must also work in the same department as Miller."

```

SELECT ename, job
FROM emp
WHERE hiredate > (
    SELECT hiredate FROM emp WHERE ename = 'JAMES'
)
AND deptno = (
    SELECT deptno FROM emp WHERE ename = 'MILLER'
);

```

Note: Practice ideas to try on your own: employees hired before any other employee in the company; the most recent hire date overall; and employees whose salary falls between the minimum and maximum salary of their own department.

14. Sample Practice Schema

Many of the example queries above use a classic HR-style schema (similar to the famous Oracle 'Scott/EMP-DEPT' schema). A slightly extended version, as noted, could look like this:

```
employee(empid, empname, job, sal, hiredate, deptid, mgrid)
department(deptid, deptname, loc, mgrid)
sal_history(historyid, empid, oldsal, newsal, chgdate)
project(projid, projname, budget)
emp_project(empid, projectid, role)
```

Using tables like these, you can practice everything covered in this document — joins, subqueries, aggregate functions, grouping, filtering, and constraints — on realistic, connected data.

15. Quick Reference Summary

15.1 SQL Statement Categories

Category	Key Commands
DDL	CREATE, ALTER, DROP, TRUNCATE, RENAME
DML	INSERT, UPDATE, DELETE
TCL	COMMIT, ROLLBACK, SAVEPOINT
DCL	GRANT, REVOKE
DQL	SELECT

15.2 Logical Order to Learn SQL Queries

- 1. SELECT and FROM — pick columns and a table
- 2. WHERE — filter rows
- 3. GROUP BY — group rows
- 4. HAVING — filter groups
- 5. ORDER BY — sort the result
- 6. LIMIT — restrict the number of rows returned
- 7. Subqueries and Joins — combine data from multiple sources

— End of Notes —